READ DATA

```
import os
import random
from google.colab import drive

drive.mount('/content/drive')
train_pneumonia_dir = '/content/drive/MyDrive/TA/Dataset/train/PNEUMONIA'
test_pneumonia_dir = '/content/drive/MyDrive/TA/Dataset/test/PNEUMONIA'
train_normal_dir = '/content/drive/MyDrive/TA/Dataset/train/NORMAL'
test_normal_dir = '/content/drive/MyDrive/TA/Dataset/test/NORMAL'
```

```
Mounted at /content/drive
```

```
def get_all_files(dirs, allowed_extensions=['.jpg', '.jpeg', '.png']):
    all_files = []
    for dir_path in dirs:
        files = [os.path.join(dir_path, f) for f in os.listdir(dir_path)
                 if not f.startswith('.') and os.path.splitext(f)[1].lower() in allowed_extensions]
        all_files.extend(files)
    return all_files

def random_select_files(file_list, subset_size):
    if subset_size > len(file_list):
        raise ValueError("Jumlah file yang diminta lebih besar daripada jumlah file yang tersedia.")
    return random.sample(file_list, subset_size)

def count_files(directory):
  return len(os.listdir(directory))
```

```
all_pneumonia_files = get_all_files([train_pneumonia_dir, test_pneumonia_dir])
all_normal_files = get_all_files([train_normal_dir, test_normal_dir])

print(f"train pneumon : {count_files(train_pneumonia_dir)}")
print(f"test pneumon : {count_files(test_pneumonia_dir)}")
print(f"train norm : {count_files(train_normal_dir)}")
print(f"test norm : {count_files(test_normal_dir)}")

print(f"pneumon : {len(all_pneumonia_files)}")
```

```
print(f"norm : {len(all_normal_files)}")
```

```
train pneumon : 3875
test pneumon : 390
train norm : 1341
test norm : 234
pneumon : 4265
norm : 1575
```

BALANCING

```
train_pneumonia_count = 3200
test_pneumonia_count = 1065
train_normal_count = 3200
test_normal_count = 1065

test_pneumonia_samples = random.sample(all_pneumonia_files, test_pneumonia_count)
test_normal_samples = random.sample(all_normal_files, test_normal_count)

pneumonia_train_pool = [f for f in all_pneumonia_files if f not in test_pneumonia_samples]
normal_train_pool = [f for f in all_normal_files if f not in test_normal_samples]

train_pneumonia_samples = random.choices(pneumonia_train_pool, k=train_pneumonia_count)
train_normal_samples = random.choices(normal_train_pool, k=train_normal_count)
```

```
print(f"train_pneumonia_samples : {len(train_pneumonia_samples)}")
print(f"test_pneumonia_samples : {len(test_pneumonia_samples)}")
print(f"train_normal_samples : {len(train_normal_samples)}")
print(f"test_normal_samples : {len(test_normal_samples)}")
```

```
train_pneumonia_samples : 3200
test_pneumonia_samples : 1065
train_normal_samples : 3200
test_normal_samples : 1065
```

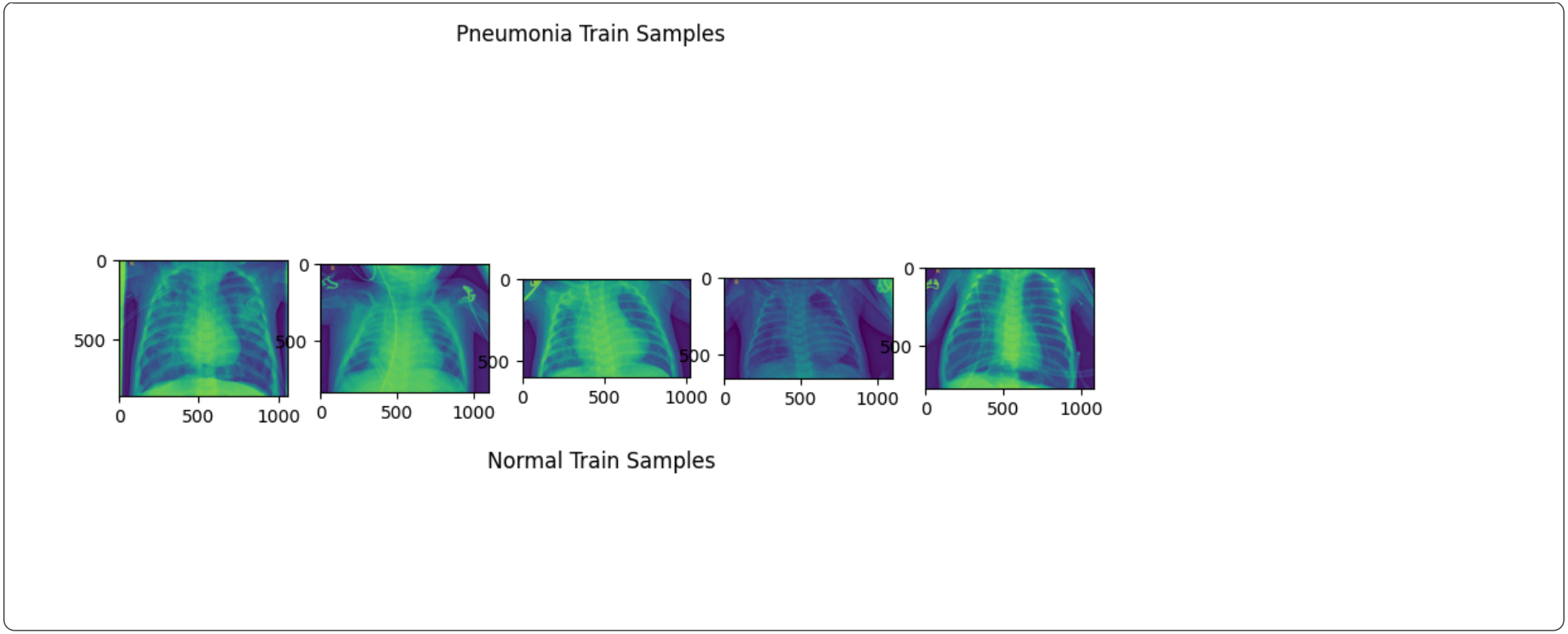
```
from PIL import Image
import matplotlib.pyplot as plt

def show_sample_images(sample_files,ammount, title):
    plt.figure(figsize=(10, 5))
    plt.suptitle(title)
    for i in range(ammount):
        plt.subplot(1, ammount, i+1)
```

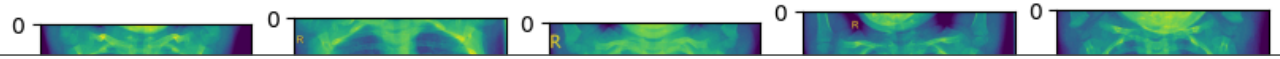
```
            img = Image.open(sample_files[i])
            plt.imshow(img)
        plt.show()

show_sample_images(train_pneumonia_samples, 5, 'Pneumonia Train Samples')
show_sample_images(train_normal_samples, 5, 'Normal Train Samples')
show_sample_images(test_pneumonia_samples, 5, 'Pneumonia Test Samples')
show_sample_images(test_normal_samples, 5, 'Normal Test Samples')
```





RESIZE



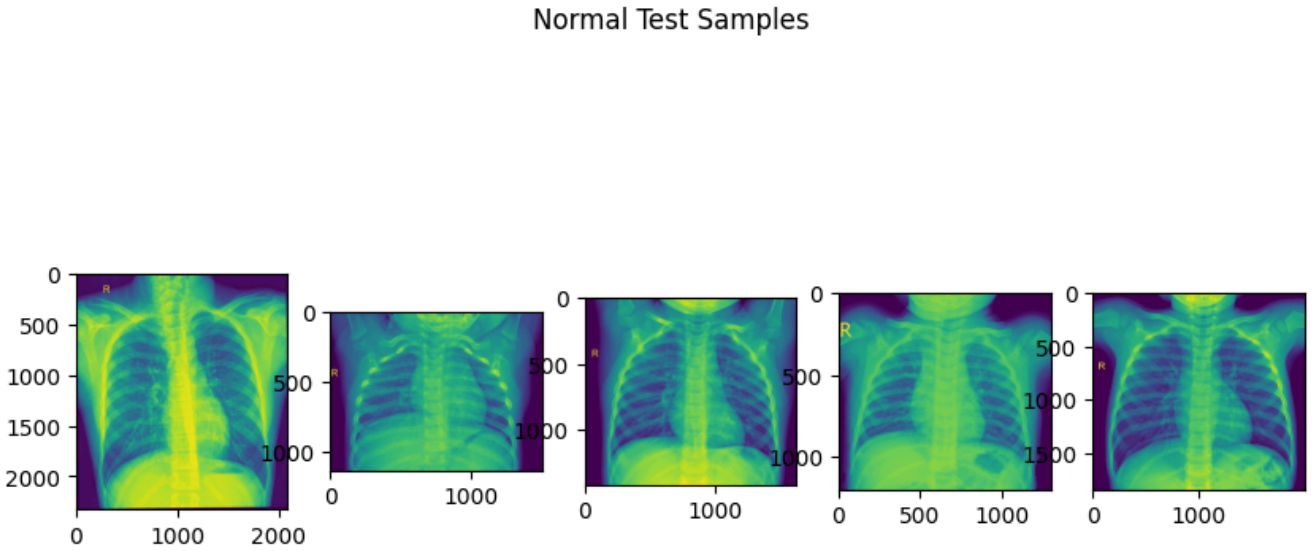
```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatten, Dense, Dropout
import numpy as np
from tensorflow.keras.preprocessing.image import load_img, img_to_array

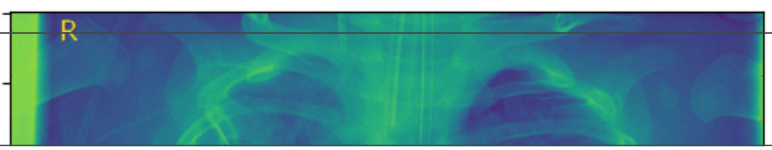
def load_images(file_paths, target_size=(224, 224)):
    images = []
    for i, file_path in enumerate(file_paths):
        img = load_img(file_path, target_size=target_size)
        img_array = img_to_array(img)
        img_array = img_array - 30
        img_array = img_array / 255.0
        images.append(img_array)
    return images

train_pneumonia_images = load_images(train_pneumonia_samples)
test_pneumonia_images = load_images(test_pneumonia_samples)
```

```
train_normal_images = load_images(train_normal_samples)
test_normal_images = load_images(test_normal_samples)
```

```
show_sample_images(train_pneumonia_samples, 1, 'before resize')
plt.imshow(train_pneumonia_images[0])
plt.title('After Resizing')
```





GLCM

```
import numpy as np
import cv2
from skimage.feature import graycomatrix, graycoprops

def calculate_glcm(image):
    gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
    distances = [1, 2, 3]
    angles = [0, np.pi/4, np.pi/2, 3*np.pi/4]

    glcm = graycomatrix(gray_image,
                        distances=distances,
                        angles=angles,
                        levels=256,
                        symmetric=True,
                        normed=True)

    properties = ['contrast', 'dissimilarity', 'homogeneity', 'energy', 'correlation', 'ASM']
    features = {}

    for prop in properties:
        values = graycoprops(glcm, prop).flatten()
        features[f'{prop}_mean'] = np.mean(values)
        features[f'{prop}_std'] = np.std(values)
        features[f'{prop}_var'] = np.var(values)

    glcm_entropy = -np.sum(glcm * np.log2(glcm + 1e-10))
    features['entropy'] = glcm_entropy

    return features
```



```
import cv2

def extract_glcm_features(image_list):
    glcm_features_list = []
    for image in image_list:
        image_uint8 = (image * 255).astype(np.uint8)
        features = calculate_glcm(image_uint8)
```

```
            glcm_features_list.append(features)
        return glcm_features_list

    train_pneumonia_glcm = extract_glcm_features(train_pneumonia_images)
    test_pneumonia_glcm = extract_glcm_features(test_pneumonia_images)
    train_normal_glcm = extract_glcm_features(train_normal_images)
    test_normal_glcm = extract_glcm_features(test_normal_images)

    print("GLCM features for the first pneumonia training image:")
    train_pneumonia_glcm[0]
```

```
GLCM features for the first pneumonia training image:
{'contrast_mean': np.float64(1215.7637282097342),
 'contrast_std': np.float64(604.2551625381589),
 'contrast_var': np.float64(365124.30145401676),
 'dissimilarity_mean': np.float64(13.446299555270656),
 'dissimilarity_std': np.float64(4.462842562958955),
 'dissimilarity_var': np.float64(19.916963741758057),
 'homogeneity_mean': np.float64(0.16560993036757773),
 'homogeneity_std': np.float64(0.04336721745535526),
 'homogeneity_var': np.float64(0.0018807155498200701),
 'energy_mean': np.float64(0.014852845892011206),
 'energy_std': np.float64(0.001976914214063014),
 'energy_var': np.float64(3.908189809764385e-06),
 'correlation_mean': np.float64(0.8109860402664245),
 'correlation_std': np.float64(0.09408777855407914),
 'correlation_var': np.float64(0.008852510073241435),
 'ASM_mean': np.float64(0.00022451522090159858),
 'ASM_std': np.float64(6.113779159230574e-05),
 'ASM_var': np.float64(3.7378295607842105e-09),
 'entropy': np.float64(153.51413650128237)}
```

```
#import pandas as pd
#from google.colab import files

#train_pneumonia_df = pd.DataFrame(train_pneumonia_glcm)
#train_pneumonia_df['label'] = 'PNEUMONIA'

#test_pneumonia_df = pd.DataFrame(test_pneumonia_glcm)
#test_pneumonia_df['label'] = 'PNEUMONIA'

#train_normal_df = pd.DataFrame(train_normal_glcm)
#train_normal_df['label'] = 'NORMAL'

#test_normal_df = pd.DataFrame(test_normal_glcm)
#test_normal_df['label'] = 'NORMAL'
```

```
#all_glcm_df = pd.concat([train_pneumonia_df, test_pneumonia_df, train_normal_df, test_normal_df], ignore_index=True)

#excel_filename = 'glcm_features.xlsx'
#all_glcm_df.to_excel(excel_filename, index=False)

#files.download(excel_filename)

#print(f"GLCM features saved to '{excel_filename}' and downloaded.")
```

CNN

```
import pandas as pd
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout, BatchNormalization
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping, ReduceLROnPlateau
from tensorflow.keras.regularizers import l2

# GLCM - DF
train_pneumonia_glcm_df = pd.DataFrame(train_pneumonia_glcm)
test_pneumonia_glcm_df = pd.DataFrame(test_pneumonia_glcm)
train_normal_glcm_df = pd.DataFrame(train_normal_glcm)
test_normal_glcm_df = pd.DataFrame(test_normal_glcm)

train_pneumonia_glcm_df['label'] = 1  # 1 for pneumonia
test_pneumonia_glcm_df['label'] = 1
train_normal_glcm_df['label'] = 0  # 0 for normal
test_normal_glcm_df['label'] = 0

train_glcm_df = pd.concat([train_pneumonia_glcm_df, train_normal_glcm_df], ignore_index=True)
test_glcm_df = pd.concat([test_pneumonia_glcm_df, test_normal_glcm_df], ignore_index=True)


# Separate features
X_train = train_glcm_df.drop('label', axis=1)
y_train = train_glcm_df['label']
X_test = test_glcm_df.drop('label', axis=1)
y_test = test_glcm_df['label']

# Normal
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
# Model
model = Sequential([
    Dense(128, activation='relu', kernel_regularizer=l2(0.0001), input_shape=(X_train.shape[1],)),
    BatchNormalization(),
    Dropout(0.3),
    Dense(64, activation='relu', kernel_regularizer=l2(0.0001)),
    BatchNormalization(),
    Dropout(0.3),
    #Dense(32, activation='relu', kernel_regularizer=l2(0.0001)),
    #BatchNormalization(),
    #Dropout(0.3),
    Dense(1, activation='sigmoid')
])

optimizer = Adam(learning_rate=0.001)
class_weights = {0: 1.5, 1: 0.8}

callbacks = [
    EarlyStopping(monitor='val_loss', patience=50, restore_best_weights=True),
    ReduceLROnPlateau(monitor='val_loss', factor=0.3, patience=25, min_lr=1e-6)
]

model.compile(optimizer=optimizer,
              loss='binary_crossentropy',
              metrics=['accuracy', tf.keras.metrics.AUC()])

model.summary()

history = model.fit(
    X_train, y_train,
    epochs=50,
    #epochs=100,
    batch_size=32,
    validation_split=0.3,
    callbacks=None,
    #callbacks=callbacks,
    class_weight=class_weights,
)
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Model: "sequential_4"
```

Layer (type)	Output Shape	Param #
dense_11 (Dense)	(None, 128)	2,560
batch_normalization_7 (BatchNormalization)	(None, 128)	512
dropout_7 (Dropout)	(None, 128)	0
dense_12 (Dense)	(None, 64)	8,256
batch_normalization_8 (BatchNormalization)	(None, 64)	256
dropout_8 (Dropout)	(None, 64)	0
dense_13 (Dense)	(None, 1)	65

```
 Total params: 11,649 (45.50 KB)
 Trainable params: 11,265 (44.00 KB)
 Non-trainable params: 384 (1.50 KB)
Epoch 1/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 5s 7ms/step - accuracy: 0.8004 - auc_4: 0.8827 - loss: 0.4738 - val_accuracy: 0.8073 - val_auc_4: 0.0000e+00 - val_loss: 0.543
Epoch 2/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.8502 - auc_4: 0.9187 - loss: 0.3807 - val_accuracy: 0.8641 - val_auc_4: 0.0000e+00 - val_loss: 0.449
Epoch 3/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.8672 - auc_4: 0.9307 - loss: 0.3450 - val_accuracy: 0.8797 - val_auc_4: 0.0000e+00 - val_loss: 0.351
Epoch 4/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.8699 - auc_4: 0.9330 - loss: 0.3408 - val_accuracy: 0.8880 - val_auc_4: 0.0000e+00 - val_loss: 0.320
Epoch 5/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.8837 - auc_4: 0.9384 - loss: 0.3246 - val_accuracy: 0.8917 - val_auc_4: 0.0000e+00 - val_loss: 0.292
Epoch 6/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.8761 - auc_4: 0.9408 - loss: 0.3174 - val_accuracy: 0.8969 - val_auc_4: 0.0000e+00 - val_loss: 0.293
Epoch 7/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8819 - auc_4: 0.9442 - loss: 0.3088 - val_accuracy: 0.8656 - val_auc_4: 0.0000e+00 - val_loss: 0.348
Epoch 8/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8855 - auc_4: 0.9473 - loss: 0.3016 - val_accuracy: 0.8948 - val_auc_4: 0.0000e+00 - val_loss: 0.282
Epoch 9/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8859 - auc_4: 0.9486 - loss: 0.2982 - val_accuracy: 0.8964 - val_auc_4: 0.0000e+00 - val_loss: 0.292
Epoch 10/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.8884 - auc_4: 0.9482 - loss: 0.2972 - val_accuracy: 0.9057 - val_auc_4: 0.0000e+00 - val_loss: 0.265
Epoch 11/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8924 - auc_4: 0.9527 - loss: 0.2861 - val_accuracy: 0.8859 - val_auc_4: 0.0000e+00 - val_loss: 0.306
Epoch 12/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8998 - auc_4: 0.9533 - loss: 0.2820 - val_accuracy: 0.9193 - val_auc_4: 0.0000e+00 - val_loss: 0.242
Epoch 13/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8944 - auc_4: 0.9542 - loss: 0.2782 - val_accuracy: 0.9146 - val_auc_4: 0.0000e+00 - val_loss: 0.241
Epoch 14/50
```

```
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.8931 - auc_4: 0.9541 - loss: 0.2816 - val_accuracy: 0.8901 - val_auc_4: 0.0000e+00 - val_loss: 0.286
Epoch 15/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.8953 - auc_4: 0.9559 - loss: 0.2758 - val_accuracy: 0.9005 - val_auc_4: 0.0000e+00 - val_loss: 0.273
Epoch 16/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.8984 - auc_4: 0.9578 - loss: 0.2689 - val_accuracy: 0.9078 - val_auc_4: 0.0000e+00 - val_loss: 0.264
Epoch 17/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9002 - auc_4: 0.9589 - loss: 0.2651 - val_accuracy: 0.9271 - val_auc_4: 0.0000e+00 - val_loss: 0.226
Epoch 18/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8991 - auc_4: 0.9599 - loss: 0.2621 - val_accuracy: 0.9078 - val_auc_4: 0.0000e+00 - val_loss: 0.257
Epoch 19/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.8946 - auc_4: 0.9561 - loss: 0.2743 - val_accuracy: 0.8833 - val_auc_4: 0.0000e+00 - val_loss: 0.280
Epoch 20/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - accuracy: 0.9029 - auc_4: 0.9599 - loss: 0.2619 - val_accuracy: 0.9083 - val_auc_4: 0.0000e+00 - val_loss: 0.252
Epoch 21/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.9047 - auc_4: 0.9598 - loss: 0.2618 - val_accuracy: 0.9292 - val_auc_4: 0.0000e+00 - val_loss: 0.208
Epoch 22/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.9022 - auc_4: 0.9619 - loss: 0.2562 - val_accuracy: 0.8932 - val_auc_4: 0.0000e+00 - val_loss: 0.283
Epoch 23/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.9047 - auc_4: 0.9620 - loss: 0.2551 - val_accuracy: 0.9323 - val_auc_4: 0.0000e+00 - val_loss: 0.247
Epoch 24/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.8987 - auc_4: 0.9628 - loss: 0.2529 - val_accuracy: 0.9167 - val_auc_4: 0.0000e+00 - val_loss: 0.242
Epoch 25/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9078 - auc_4: 0.9622 - loss: 0.2544 - val_accuracy: 0.9083 - val_auc_4: 0.0000e+00 - val_loss: 0.246
Epoch 26/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9045 - auc_4: 0.9617 - loss: 0.2557 - val_accuracy: 0.9099 - val_auc_4: 0.0000e+00 - val_loss: 0.259
Epoch 27/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9060 - auc_4: 0.9619 - loss: 0.2560 - val_accuracy: 0.8995 - val_auc_4: 0.0000e+00 - val_loss: 0.270
Epoch 28/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9071 - auc_4: 0.9655 - loss: 0.2418 - val_accuracy: 0.9167 - val_auc_4: 0.0000e+00 - val_loss: 0.238
Epoch 29/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9098 - auc_4: 0.9645 - loss: 0.2481 - val_accuracy: 0.9062 - val_auc_4: 0.0000e+00 - val_loss: 0.262
Epoch 30/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9031 - auc_4: 0.9634 - loss: 0.2484 - val_accuracy: 0.9245 - val_auc_4: 0.0000e+00 - val_loss: 0.252
Epoch 31/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9047 - auc_4: 0.9656 - loss: 0.2423 - val_accuracy: 0.9307 - val_auc_4: 0.0000e+00 - val_loss: 0.223
Epoch 32/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9058 - auc_4: 0.9676 - loss: 0.2376 - val_accuracy: 0.9156 - val_auc_4: 0.0000e+00 - val_loss: 0.252
Epoch 33/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9067 - auc_4: 0.9652 - loss: 0.2458 - val_accuracy: 0.9229 - val_auc_4: 0.0000e+00 - val_loss: 0.214
Epoch 34/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9096 - auc_4: 0.9660 - loss: 0.2416 - val_accuracy: 0.9042 - val_auc_4: 0.0000e+00 - val_loss: 0.247
Epoch 35/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9121 - auc_4: 0.9675 - loss: 0.2367 - val_accuracy: 0.9271 - val_auc_4: 0.0000e+00 - val_loss: 0.235
Epoch 36/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9089 - auc_4: 0.9664 - loss: 0.2403 - val_accuracy: 0.9229 - val_auc_4: 0.0000e+00 - val_loss: 0.242
Epoch 37/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9098 - auc_4: 0.9649 - loss: 0.2456 - val_accuracy: 0.9484 - val_auc_4: 0.0000e+00 - val_loss: 0.196
Epoch 38/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9132 - auc_4: 0.9655 - loss: 0.2420 - val_accuracy: 0.9266 - val_auc_4: 0.0000e+00 - val_loss: 0.223
Epoch 39/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - accuracy: 0.9096 - auc_4: 0.9680 - loss: 0.2343 - val_accuracy: 0.9250 - val_auc_4: 0.0000e+00 - val_loss: 0.225
Epoch 40/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - accuracy: 0.9145 - auc_4: 0.9687 - loss: 0.2308 - val_accuracy: 0.9438 - val_auc_4: 0.0000e+00 - val_loss: 0.197
```

```
from sklearn.metrics import precision_score, recall_score
y_pred = model.predict(X_test)
y_pred_classes = (y_pred > 0.5).astype(int)
test_loss, test_accuracy, test_auc = model.evaluate(X_test, y_test, verbose=0)
test_precision = precision_score(y_test, y_pred_classes)
test_recall = recall_score(y_test, y_pred_classes)

print(f"- Loss: {test_loss:.4f}")
print(f"- AUC: {test_auc:.4f}")
print(f"- Accuracy: {test_accuracy:.4f}")
```

```
Epoch 47/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step
Epoch 48/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - accuracy: 0.9132 - auc_4: 0.9705 - loss: 0.2255 - val_accuracy: 0.9333 - val_auc_4: 0.0000e+00 - val_loss: 0.220
Epoch 49/50
140/140 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - accuracy: 0.9096 - auc_4: 0.9697 - loss: 0.2278 - val_accuracy: 0.9401 - val_auc_4: 0.0000e+00 - val_loss: 0.213
Epoch 50/50
67/67 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - accuracy: 0.9167 - auc_4: 0.9770 - loss: 0.2138 - val_accuracy: 0.9333 - val_auc_4: 0.0000e+00 - val_loss: 0.2988
- Loss: 0.2988
- AUC: 0.9770
- Accuracy: 0.8930
```

```
from sklearn.metrics import classification_report
print(classification_report(y_test, y_pred_classes))
```

```
              precision    recall  f1-score   support

           0       0.91      0.87      0.89      1065
           1       0.88      0.92      0.90      1065

    accuracy                           0.89      2130
   macro avg       0.89      0.89      0.89      2130
weighted avg       0.89      0.89      0.89      2130
```

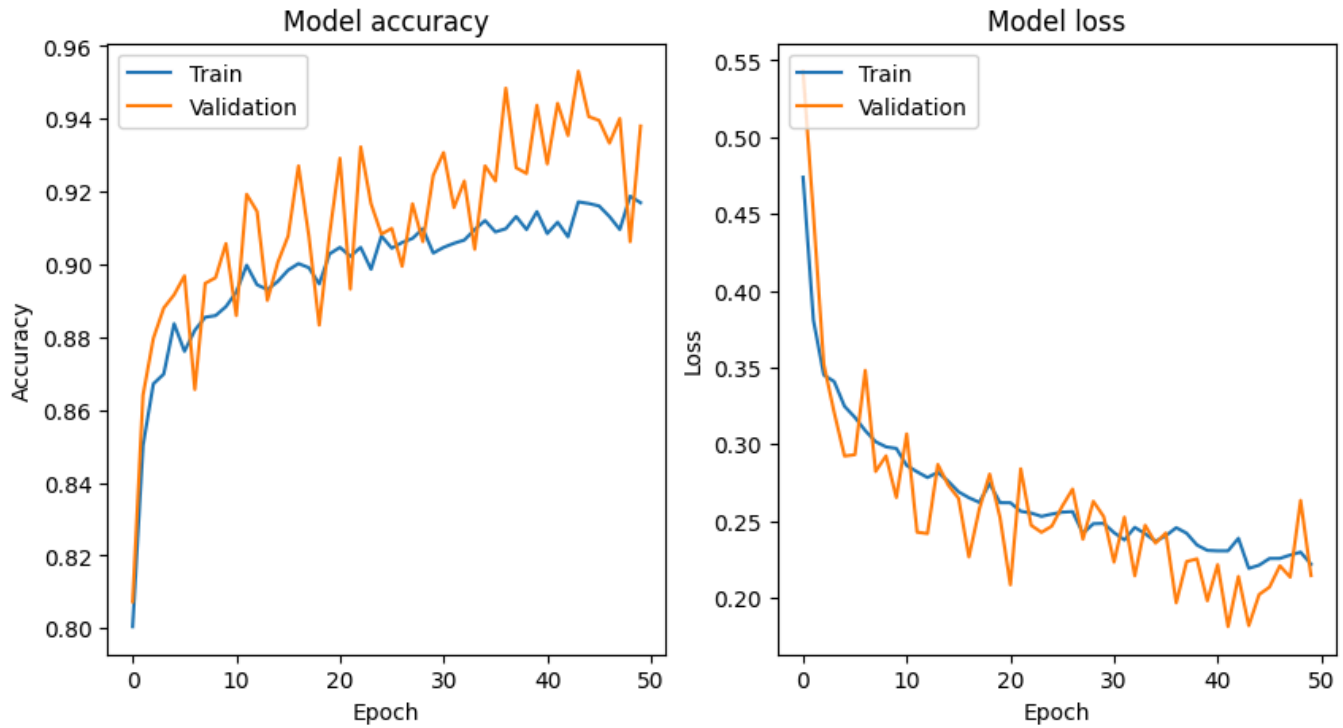
```
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 5))
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('Model accuracy')
plt.ylabel('Accuracy')
plt.xlabel('Epoch')
plt.legend(['Train', 'Validation'], loc='upper left')

plt.subplot(1, 2, 2)
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('Model loss')
plt.ylabel('Loss')
plt.xlabel('Epoch')
```

```
plt.legend(['Train', 'Validation'], loc='upper left')

plt.show()
```



PREDICT

```
y_pred = model.predict(X_test)
y_pred_classes = (y_pred > 0.5).astype(int)
```

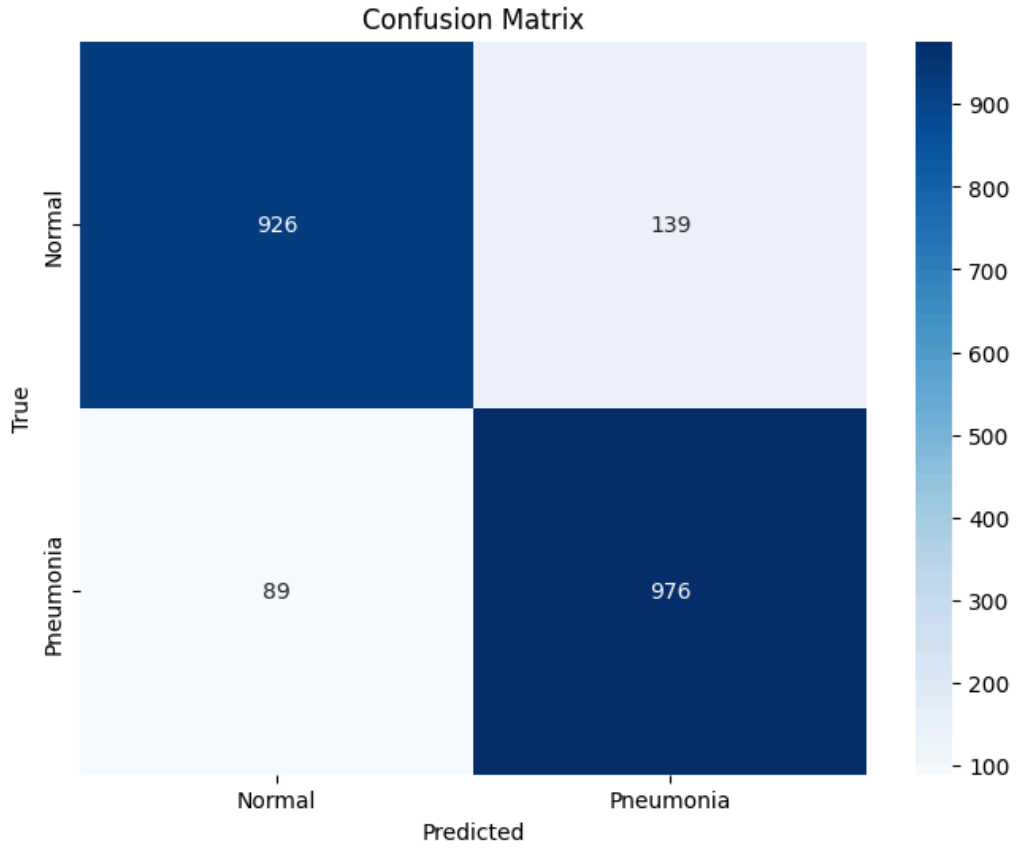
```
67/67 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step
```

```
from sklearn.metrics import confusion_matrix
import seaborn as sns

conf_matrix = confusion_matrix(y_test, y_pred_classes)

plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Normal', 'Pneumonia'], yticklabels=['Normal', 'Pneumonia'])
```

```
plt.xlabel('Predicted')
plt.ylabel('True')
plt.title('Confusion Matrix')
plt.show()
```




```
model.save("Pnem_sim.h5")
```

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format is considered legacy. We re
```

```
from google.colab import files
files.download("Pnem_sim.h5")
```

```
import pickle

with open('scaler.pkl', 'wb') as f:
    pickle.dump(scaler, f)
```

```
files.download("scaler.pkl")
```

CODE END HERE

Tes SVM

```
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Create and train the SVM model
svm_model = SVC(kernel='linear', random_state=42) # You can choose different kernels
svm_model.fit(X_train, y_train)

# Make predictions on the test set
y_pred_svm = svm_model.predict(X_test)

# Evaluate the SVM model
print("\n--- SVM Model Evaluation ---")
print(f"Accuracy: {accuracy_score(y_test, y_pred_svm):.4f}")
print("\nClassification Report:")
print(classification_report(y_test, y_pred_svm))

# Confusion Matrix
conf_matrix_svm = confusion_matrix(y_test, y_pred_svm)
plt.figure(figsize=(8, 6))
sns.heatmap(conf_matrix_svm, annot=True, fmt='d', cmap='Blues',
            xticklabels=['Normal', 'Pneumonia'], yticklabels=['Normal', 'Pneumonia'])
plt.xlabel('Predicted')
plt.ylabel('True')
plt.title('Confusion Matrix (SVM)')
plt.show()
```